

Managing Secrets in Kubernetes with Sealed Secrets

'Secrets' in Kubernetes cannot be stored in a Git repository as that would expose confidential and sensitive information to everyone who can access this repository. 'Sealed Secrets' has been developed to address this specific challenge.



When deploying an application in Kubernetes, resources are generally described in a group of *yaml* files and stored in a Git repository. This works for all of the Kubernetes resources except 'secrets', as that exposes sensitive information to everyone having access to the repository.

'Sealed Secrets' is a controller developed by Bitnami to solve this particular problem. With Sealed Secrets in place, you can manifest it in your Git repository, which will be automatically decrypted by the controller running in your cluster.

Installation

There are various ways to install Sealed Secrets in your Kubernetes cluster: Kustomize, Helm Chart or via the Operator framework.

I prefer Kustomize as you can just use plain *kubectl* to perform the installation. Let's create a kustomization file for Sealed Secrets:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
```

```
namespace: kube-system
```

```
resources:
- resources.yaml
```

The *resources.yaml* file contains the resources related to Sealed Secrets. Let's apply it:

```
> kubectl apply -k sealed-secrets/
customresourcedefinition.apiextensions.k8s.io/sealedsecrets.
bitnami.com created
serviceaccount/sealed-secrets-controller created
role.rbac.authorization.k8s.io/sealed-secrets-key-admin
created
role.rbac.authorization.k8s.io/sealed-secrets-service-proxier
created
clusterrole.rbac.authorization.k8s.io/secrets-unsealer created
rolebinding.rbac.authorization.k8s.io/sealed-secrets-
controller created
rolebinding.rbac.authorization.k8s.io/sealed-secrets-service-
proxier created
clusterrolebinding.rbac.authorization.k8s.io/sealed-secrets-
controller created
service/sealed-secrets-controller created
deployment.apps/sealed-secrets-controller created
```

Usage

Now let's say that you have the following secret resource:

```
apiVersion: v1
kind: Secret
```